Computer Systems Modelling: *Simulation*

Dr RJ Gibbens
Computer Science Tripos, Part II
Lent Term 2020/21

Last revised: 2018-01-10 (34a98a7)
Rearranged: 2020-12-23

Department of Computer Science & Technology
The Computer Laboratory
University of Cambridge

Simulation

Simulation allows arbitrarily complex systems to be evaluated.

- Able to capture the dynamic behaviour of systems.
- The dynamics of complex systems are obtained by tracking the evolution of the system over time.

Examples include communication network design, road traffic modelling, etc.



Types of simulation

- Discrete state/event simulation in which the state of the system is described by discrete variables

  — e.g. the number of jobs at different stages on a production line

- Continuous state/event simulation in which the state is described by continuous variables

  — e.g. the quantities of various chemical reagents

A similar distinction may be drawn between discrete time and continuous time simulations depending on whether the system state is defined at certain discrete times or at all times.

Types of simulation (2)

We will be concerned with discrete event simulation because it applies most naturally to computer systems in which state variables are generally discrete. For example:

- the state of jobs in the system;
- the number of jobs of different kinds;
- the number or availability of resources or devices.

State space

Simulation is of particular use when we are studying systems that are not in steady state.

Execution of a simulation model consists of a series of state space changes.

We always follow the dynamic evolution of the system, even if we only want a mean value. Each execution of the dynamic system results in a trajectory.



Pros and cons

The principal advantage of simulation is its extreme generality. However, ...

- The design, coding and debugging of a simulation program are often time consuming and difficult to understand — the task may even approach that of implementing the system and measuring it directly!
- Generality can lead to complexity that can obscure understanding of the model — fine details may be irrelevant if the workload is poorly approximated.
- Execution of the simulation can be computationally expensive.
- Statistical analysis of the output can be problematic — e.g. how long to run the simulation before averaging the results?



Simulation loop

Each event contains a time stamp identifying 'when it occurs' and denotes some change in the state of the system to be simulated e.g. 'IP packet arrived'

Events are ordered in time in an event list

1: Initialize the clock to 0
2: Initialize the event list
3: **while** termination criterion is not met **do**
4: remove earliest tuple (t, m) from the event list
5: update the clock to t
6: simulate the effect of transmitting m at time t
7: **end while**



It is crucial that the simulator always selects the event with the earliest time stamp.

Frequently most of the simulation time is spent maintaining the chronological order of the event list.

Simulation variables

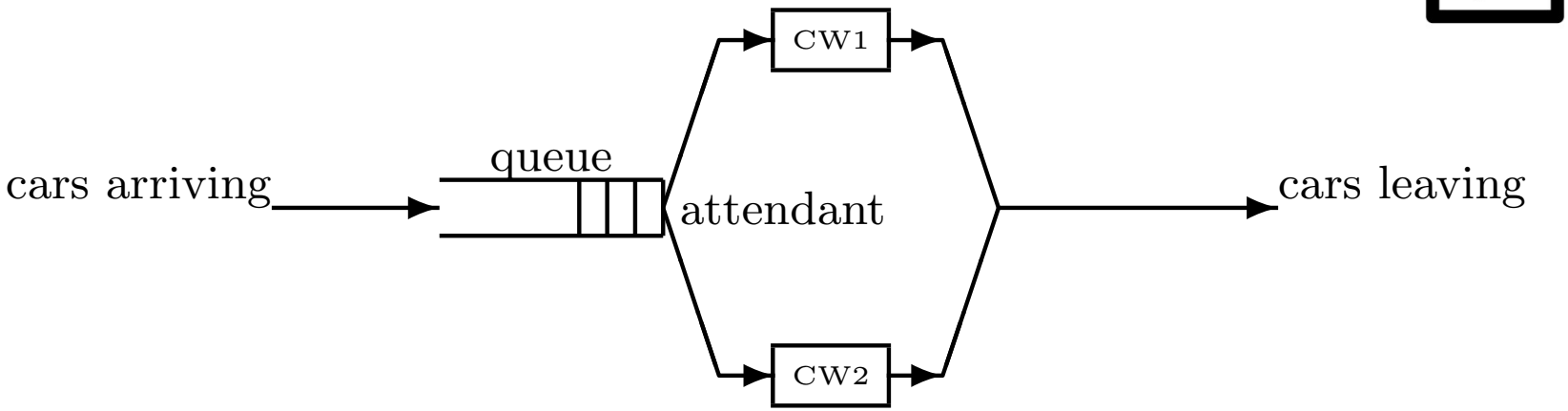
As we have seen, discrete event simulations involve both events and variables and where we keep track of variables as they change during a simulation.

There are several types of variables to consider.

- Time variable, t, to record the passage of time during the simulation;
- Counter variables which keep count of the number of times that certain events have occurred by time t;
- System state variables which define the state of the system at time t.

As simulation events occur we change these variables and gather any output of interest.

Example



A car wash which is able to service one car at a time — is it viable to install a second car wash?

Model: attendant (**att**), car washes, (**cw1**, **cw2**), entrance (**source**) and exit (**sink**).

- The inter-arrival time of cars that need to be washed is randomly distributed.
- If both car washes are busy then an arriving car joins the queue.
- When a car wash becomes idle then the car at the head of the queue is sent to it.

Example (2)

Events:

- **source→att**: car arrives in the system
- **att→cw**: car sent from the queue to the car wash
- **cw→att**: car wash becomes idle
- **cw→sink**: car departs the system

Note how the departure of a car is modelled by two events: one representing the car leaving the system and the other that signals to the attendant that the car wash is now idle. Observe the similarity with object-oriented styles of programming in which objects communicate solely by method invocations.

Given that it takes **cw1** 8 minutes and **cw2** 10 minutes to wash a car, a possible sequence of events is ...

Sequence of events

message	event	time	sender	receiver	content
1	-	0	cw1	att	idle
2	-	0	cw2	att	idle
3	1	6	source	att	car 1
4	2	6	att	cw1	car 1
5	3	11	source	att	car 2
6	4	11	att	cw2	car 2
7	5	12	source	att	car 3
8	6	14	cw1	sink	car 1
9	-	14	cw1	att	idle
10	7	14	att	cw1	car 3
11	8	17	source	att	car 4
12	9	19	source	att	car 5
13	10	21	cw2	sink	car 2
14	-	21	cw2	att	idle

Random number generation



Watch this first!

It is important not to generate random numbers with an ad hoc method — complex algorithms do not necessarily generate random outputs.

Some operating systems include support for generating random numbers based on (e.g.) key strokes or network inter-arrival times — however, these mechanisms are not usually suited to generating large volumes of random data.

How can we algorithmically generate a long random sequence?

The answer is that the sequence is not random, but appears random as far as can be determined from statistical tests. Such a sequence is termed pseudo-random.

Random number generation (2)

Important requirements include:

- the algorithm should be fast;
- the storage requirements should be low;
- the random sequence should only repeat after a very long period;
- the sequence generated should possess two important statistical properties: uniformity and independence.

For ease of implementation and speed most random number generators use an integer representation over an interval $[0, m)$.

Given a value in $[0, m)$ a value in the range $[0, 1)$, say, can be obtained by dividing by m



Random variable generation

We assume that we have an efficiently generated supply of random numbers distributed as independent $U(0,1)$ random variables.

How do we now generate variables from other distributions?

We will find that with a suitable transformation the resulting random variables will appear to have been drawn from any desired probability distribution.

Discrete distributions

Suppose we are given a distribution p_i with $0 \leq p_i \leq 1$ for $i \in I = \{0, 1, \ldots\}$ and $\sum_{i \in I} p_i = 1$ and that we wish to generate a discrete random variable, X, whose probability distribution function is

$$\mathbb{P}(X = x_i) = p_i \qquad \forall i \in I\,.$$

This may be done by generating a (pseudo-)random variable U with distribution $U(0, 1)$ and setting

$$X = \begin{cases} x_0 & \text{if } U < p_0 \\ x_1 & \text{if } p_0 \leq U < p_0 + p_1 \\ x_2 & \text{if } p_0 + p_1 \leq U < p_0 + p_1 + p_2 \\ \vdots & \\ x_i & \text{if } \sum_{j=0}^{i-1} p_j \leq U < \sum_{j=0}^{i} p_j \\ \vdots & \end{cases}$$



The inverse transform method

For now since U is $U(0,1)$

$$\mathbb{P}(X = x_i) = \mathbb{P}\left(\sum_{j=0}^{i-1} p_j \leq U < \sum_{j=0}^{i} p_j\right) = p_i$$

If we write $F(x_k) = \sum_{i=0}^{k} p_i$ then the process of generating X is given by

- Generate a random number U
- If $U < F(x_0)$ set $X = x_0$ and stop
- If $U < F(x_1)$ set $X = x_1$ and stop
- $\vdots$

If the x_i are ordered so that $x_0 < x_1 < \cdots$ then this is equivalent to choosing $X = x_i$ if $F(x_{i-1}) \leq U < F(x_i)$ and for this reason the method is known as the inverse transform method.

Geometric random variables

Here

$$p_i = \mathbb{P}(X = i) = p(1-p)^{i-1} \quad i = 1, 2, \ldots$$

and so

$$\begin{aligned}\sum_{j=1}^{i-1} p_j &= 1 - \mathbb{P}(X > i-1) \\ &= 1 - (1-p)^{i-1}\,.\end{aligned}$$

Thus, we can use the inverse transform method by setting X to the value of i such that

$$1 - (1-p)^{i-1} \leq U < 1 - (1-p)^{i}\,.$$

A little algebra shows that we can write this as

$$X = \left\lfloor \frac{\log(U)}{\log(1-p)} \right\rfloor + 1\,.$$

Poisson random variables

Here we have for $\lambda > 0$

$$p_i = \mathbb{P}(X = i) = e^{-\lambda} \frac{\lambda^i}{i!} \quad i = 0, 1, \ldots .$$

Hence, it follows that

$$p_{i+1} = \frac{\lambda}{i+1} p_i \quad i = 0, 1, \ldots$$

and an algorithm to generate a Poisson(λ) random variable is as follows:

1. Generate a random number U
2. Set $i = 0, p = e^{-\lambda}, F = p$
3. If $U < F$, set $X = i$ and stop
4. Set $p = \lambda p/(i+1), F = F + p, i = i + 1$
5. Go to step 3

Similar algorithms can be formulated for other discrete distributions.

Generating continuous random variables

Let U be a random variable with distribution $U(0,1)$ then

$$X = F_X^{-1}(U)$$



is a random variable with distribution function $F_X(x)$.
Proof:

$$\begin{aligned}
\mathbb{P}(X \leq x) &= \mathbb{P}(F_X^{-1}(U) \leq x) \\
&= \mathbb{P}(F_X(F_X^{-1}(U)) \leq F_X(x)) \\
&= \mathbb{P}(U \leq F_X(x)) \\
&= F_X(x)\,.
\end{aligned}$$

Proof video is in 2 parts





So, the inverse transform method for continuous random variables with distribution function $F_X(x)$ generates $X = F_X^{-1}(U)$ where U is $U(0,1)$.

Uniform distribution (a, b)

Consider the uniform random variable on the interval (a, b) with distribution function

$$F_X(x) = (x - a)/(b - a)$$

for x in the interval (a, b).

Given a pseudo-random uniform value U in the interval $(0, 1)$ we set

$$X = F_X^{-1}(U)$$

and so

$$U = F_X(X) = (X - a)/(b - a)$$

so that

$$X = (b - a)U + a\,.$$

Exponential distribution

For the exponential distribution with parameter λ we have

$$F_X(x) = \begin{cases} 1 - e^{-\lambda x} & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$$

and so given U, a pseudo-random variable with distribution $U(0,1)$, we set

$$X = F_X^{-1}(U)\,.$$

Thus



$$U = F_X(X) = 1 - e^{-\lambda X}$$

so that

$$X = -\frac{1}{\lambda}\log(1-U)\,.$$

Note that both U and $1-U$ are distributed as $U(0,1)$ so we might as well set

$$X = -\frac{1}{\lambda}\log(U)\,.$$

Simulating a Poisson process

Consider the problem of generating the first n event times of a Poisson process of rate λ. One way is to generate $U_1, U_2, \ldots, U_n$ independent random numbers each from a $U(0,1)$ distribution and set $X_i = -\frac{1}{\lambda}\log(U_i)$. Then the first n event times are $\sum_{i=1}^{j} X_i$ for $j = 1, 2, \ldots, n$.

To generate the first T time units we could proceed as above and stop when the sum first exceeds T. Algorithmically,

1. Set $t = 0, I = 0$
2. Generate a random number U
3. Set $t = t - \frac{1}{\lambda}\log(U)$. If $t > T$ stop
4. Set $I = I + 1, S(I) = t$
5. Go to step 2

will build the sequence of event times in $S(\cdot)$.

Simulation performance measures

As the simulation itself is stochastic so too are the observed outputs.

It is critical to realize that a simulation can only yield estimates for performance measures and there will always be some statistical error in the estimates.

We can attempt to reduce the error by:

- running the simulation for longer until sufficient samples have been taken;
- running the same simulation with a different pseudo-random number sequence, and combining the results from multiple runs. We will say more on this later.

Performance metrics

- Utilization: The utilization is the proportion of time that a server is busy.
  An estimate can therefore be obtained by taking the sum of the busy times of the server and dividing by T, the simulation length.
  In the case of a k-server, the busy times can be estimated together and divided by kT.
- Throughput: mean number of customers receiving service per unit time
- Mean queueing time
- Mean waiting time



Statistical analysis of results

Suppose that $X_1, X_2, \ldots, X_n$ are independent, identically distributed random variables and let μ and σ^2 be their common mean and variance, respectively.

Hence,

$$\mathbb{E}(X_i) = \mu$$



and

$$\mathrm{Var}(X_i) = \mathbb{E}((X_i - \mathbb{E}(X_i))^2) = \sigma^2 \,.$$

Given a sample of values of the random variables $X_1, X_2, \ldots, X_n$ how might we estimate the values of the true but unknown parameters μ and σ^2 and how does the sample size, n, effect the accuracy of our estimates?

Sample mean

The quantity

$$\overline{X}_n := \frac{1}{n}\sum_{i=1}^{n} X_i$$

is called the sample mean.

Note that the sample mean is also a random variable with expected value given by

$$\begin{aligned}
\mathbb{E}(\overline{X}_n) &= \mathbb{E}\left(\sum_{i=1}^{n} \frac{X_i}{n}\right) \\
&= \sum_{i=1}^{n} \frac{\mathbb{E}(X_i)}{n} \\
&= \frac{n\mu}{n} \\
&= \mu\,.
\end{aligned}$$

Sample mean (2)

Similarly, the variance of $\overline{X}_n$ is given by

$$\begin{aligned}
\mathrm{Var}(\overline{X}_n) &= \mathbb{E}((\overline{X}_n - \mu)^2) \\
&= \mathrm{Var}\left(\sum_{i=1}^{n} \frac{X_i}{n}\right) \\
&= \frac{1}{n^2} \sum_{i=1}^{n} \mathrm{Var}(X_i) \\
&= \frac{\sigma^2}{n} \,.
\end{aligned}$$

Thus, $\overline{X}_n$ can be used to estimate μ from the sample. It is a good estimator of μ when $\sigma/\sqrt{n}$ is small.

We can view $\mathrm{Var}(\overline{X}_n)$ as the mean squared error in our estimation of μ by $\overline{X}_n$.

Sample variance

Call the random variable

$$S_n^2 := \frac{\sum_{i=1}^{n}(X_i - \overline{X}_n)^2}{n-1}$$

the sample variance.



It is possible to show that

$$\mathbb{E}(S_n^2) = \sigma^2$$

and so S_n can be used to estimate the true standard deviation $\sqrt{\mathrm{Var}(X_i)} = \sigma$.

It is important to have an estimator for σ since we have seen that the accuracy of our estimator $(\overline{X}_n)$ for the true mean μ depends on the variance σ^2 as well as on the sample size, n.



Confidence intervals

We can use the Central Limit Theorem to see that for large sample sizes, n, the random variable

$$\frac{(\overline{X}_n - \mu)}{\sigma/\sqrt{n}} = \sqrt{n}\frac{(\overline{X}_n - \mu)}{\sigma}$$

is asymptotically distributed $N(0,1)$ for large n.

Additionally, we usually do not know the true standard deviation σ but instead we can estimate it by S_n so that

Central Limit theorem



$$\sqrt{n}\frac{(\overline{X}_n - \mu)}{S_n}$$

is approximately distributed $N(0,1)$.

Confidence intervals (2)

Write z_α for the value such that $\mathbb{P}(Z > z_\alpha) = \alpha$ where Z is a standard normal random variable $N(0,1)$ then it follows that

$$\mathbb{P}\left(-z_{\alpha/2} < Z < z_{\alpha/2}\right) = 1 - \alpha$$

and so by the CLT for large n

$$\mathbb{P}\left(-z_{\alpha/2} < \sqrt{n}\frac{(\overline{X}_n - \mu)}{S_n} < z_{\alpha/2}\right) \approx 1 - \alpha$$



or, equivalently,

$$\mathbb{P}\left(\overline{X}_n - z_{\alpha/2}\frac{S_n}{\sqrt{n}} < \mu < \overline{X}_n + z_{\alpha/2}\frac{S_n}{\sqrt{n}}\right) \approx 1 - \alpha\,.$$

The interval

$$\left[\overline{X}_n - z_{\alpha/2}S_n/\sqrt{n}, \quad \overline{X}_n + z_{\alpha/2}S_n/\sqrt{n}\right]$$

is an (approximate) $100(1-\alpha)$ percent confidence interval for μ.

Stopping rules

How do we know when a system has been run 'long enough' for performance measures to be accurate?

We can repeat the simulation several times with different random seed values to obtain many samples. These multiple runs are called replications.

Having repeated the experiment n times, we can construct a confidence interval on our measure L, say.

Although large numbers of replications reduce the variance, each replication requires re-stabilizing the simulation.

Can we avoid this?

We may be able to use a single, long run, and break up our samples into n blocks, each of these can form a sample L_i for $i = 1, 2, \ldots, n$.

Stopping rules (2)

What could go wrong with this?

Correlation between successive blocks could mean that we have biased samples.

If the block size is large then the correlation should be small.

Explicit techniques exist to estimate the correlation and determine a suitable block size.

The simulation can be stopped once the estimate of L becomes stable. For example, once the confidence interval around L becomes sufficiently narrow.

Stopping rules (3)

For example, suppose we wish to run the simulation until our $100(1-\alpha)$ confidence interval for the true mean value is of at most of width ℓ, say.

We can guarantee this by means of the following algorithm:

- generate at least 100 data values, say;
- repeatedly, generate additional data values, stopping when the number of values generated, n, is such that $2z_{\alpha/2}S_n/\sqrt{n} < \ell$.

The initial 100 data values is for illustration, a suitable value will depend on the precise simulation experiment under consideration. The intention is to suppress the effects of the initial transient phase.

Variance reduction techniques

So far we have used the sample mean, $\overline{X}_n$, as our estimator for μ, the mean value of our distribution. We know that

$$\mathbb{E}(\overline{X}_n) = \mu \qquad \text{and} \quad \text{Var}(\overline{X}_n) = \frac{\sigma^2}{n}$$



where n is the sample size.

Might we be able to find an alternative estimator for μ that has smaller variance?

Such variance reduction techniques can sometimes produce significant speed-ups in the simulation.

Antithetic variables

Suppose that X_1 and X_2 are two identically distributed random variables with common mean $\mathbb{E}(X_1) = \mathbb{E}(X_2) = \mu$. Then

$$\text{Var}\left(\frac{X_1 + X_2}{2}\right) = \frac{1}{4}\left(\text{Var}(X_1) + \text{Var}(X_2) + 2\text{Cov}(X_1, X_2)\right) .$$

Hence, we would get a reduced variance by using $(X_1 + X_2)/2$ when X_1 and X_2 are negatively correlated.

Recall that in the inverse transform method we generate (pseudo) random numbers by first generating pseudo random numbers $U_1, U_2, \ldots$ with a $U(0, 1)$ distribution. But then the numbers $1 - U_1, 1 - U_2, \ldots$ are also random numbers with a $U(0, 1)$ distribution and these two series of numbers are negatively correlated.

Antithetic variables (2)

It often happens in practice that dividing the simulation runs into two groups and using $1 - U$ for the second group in place of U in the first group yields two random variables X_1 and X_2 which are negatively correlated.

In this case we say that X_1 and X_2 are antithetic variables.

Example

Consider a queueing system and let D_i be the delay in the queue of the ith customer and suppose we wish to estimate

$$\mathbb{E}(D_1 + D_2 + \cdots + D_n)$$

the sum of the delays of the first n customers.

We should expect to require a collection of $2n$ random variables U_j (one for each arrival and departure event per customer).

Repeating the simulation using the $2n$ random numbers given by $1 - U_j$ will then give an improved estimator compared to using a 'fresh' set of $2n$ random numbers.

Additional video-only content

Some issues to watch out for when running simulations 

Simulating a controlled (closed) system 